

Tarea 2: Informe de Diseño del Modelo Lógico - Red Social Estudiantil

Lethy Stefania Valderrama Reyes

1. Contextualización

Hemos identificado una necesidad clara: mejorar la comunicación y la interacción entre los estudiantes. Aunque tenemos plataformas institucionales, a menudo son formales y no facilitan la conexión casual, el intercambio de conocimientos entre pares, la formación de grupos de estudio o la organización de eventos informales. La comunidad educativa necesita una plataforma donde puedan.

- **Crear perfiles:** compartir su área de estudio, intereses, habilidades en diferentes disciplinas o deportes, etc. //tipo de Habilidades.
- **Conectar con compañeros:** encontrar y seguir a otros estudiantes con intereses similares o de cursos avanzados que puedan ofrecer mentoría.
- **Publicar actualizaciones:** compartir preguntas sobre tareas, recursos útiles, noticias de la industria tecnológica, o incluso memes relevantes para la vida universitaria. //tipo de publicaciones.
- **Crear y unirse a grupos:** organizar grupos de estudio para materias específicas, equipos para hackatones, o clubes de interés (videojuegos, programación competitiva, etc.). //Tipos de grupos
- **Programar eventos:** anunciar y coordinar reuniones de estudio, talleres, o actividades sociales. //Tipo de eventos

2. Explicación sobre Elecciones de Nombres y Tipos de Datos

Para mantener la consistencia y claridad del diseño, tome las siguientes decisiones:

- **Nombres en singular:** Nombramos las tablas en singular (ej. ESTUDIANTE en vez de ESTUDIANTES) para representar la entidad correctamente y buenas prácticas.
- **Claves Primarias (PK):** Decidí definir como estándar el uso del prefijo id_ seguido del nombre de la tabla (ej. id_estudiante) en formato entero (INT).
- **Tipos de datos:** Utilice VARCHAR para textos ajustando la longitud según el contenido (50 para nombres, 255-1000 para descripciones y contenidos largos), y DATETIME para cualquier tipo de fecha.

3. Diccionario de Datos (Tablas Lógicas Principales)

A continuación se detallan algunas de las tablas más representativas del modelo según lo diagramado.

Tabla: ESTUDIANTE

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_estudiante	INT	Clave primaria del estudiante.	Entero autonumérico ideal para relaciones y eficiencia en búsquedas.
nombre	VARCHAR(50)	Nombre de pila.	Longitud suficiente para almacenar un nombre estándar sin desperdiciar memoria.
apellido	VARCHAR(50)	Apellidos del estudiante.	Espacio adecuado para apellidos compuestos.
correo_institucional	VARCHAR(100)	Correo de la universidad.	Permite validar el dominio universitario.
password_hash	VARCHAR(255)	Contraseña encriptada.	Necesita una longitud amplia para almacenar de forma segura los hashes de encriptación.

Tabla: PERFIL

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_perfil	INT	PK del perfil.	Identificador único del registro de perfil.
id_estudiante	INT	FK conectada a Estudiante.	Relaciona de manera única al dueño del perfil (1:1).
biografia	VARCHAR(500)	Texto de presentación.	Se asignan 500 caracteres para permitir al usuario presentarse libremente.
semestre	INT	Nivel o semestre actual.	Numérico simple que facilita ordenar o filtrar estudiantes por nivel.

Tabla: PUBLICACION

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_publicacion	INT	PK del post.	Identificador único de la publicación.
id_estudiante	INT	FK del autor del post.	Vincula la publicación con el estudiante que la creó.
contenido	VARCHAR(1000)	Cuerpo de la publicación.	Es extenso para permitir preguntas de código, reseñas o recursos largos.

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
fecha_publicacion	DATETIME	Fecha y hora del post.	Indispensable para organizar el "feed" de la red social de manera cronológica.

Tabla: GRUPO

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_grupo	INT	PK del grupo de estudio o interés.	Identificador primario.
nombre	VARCHAR(100)	Nombre del grupo.	Permite títulos descriptivos como "Grupo de Algoritmia Avanzada".
id_creador	INT	FK del estudiante fundador.	Define un administrador inicial o responsable del grupo.

4. Diccionario de Relaciones (Implementación Lógica)

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Estudiante tiene Perfil	ESTUDIANTE, PERFIL	1:1	Se agrega id_estudiante como FK en la tabla PERFIL.	Mantiene la tabla Estudiante ligera (solo datos de acceso y registro) y mueve la información opcional (biografía, foto) a Perfil.

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Estudiante pertenece a Grupo	ESTUDIANTE, GRUPO	N:M	Creación de tabla intermedia MIEMBRO_GRUPO con las llaves primarias de ambas tablas.	Cumple la regla de normalización evitando datos multivaluados. Permite que un usuario esté en varios grupos y el grupo tenga varios usuarios.
Estudiante Sigue a Estudiante	ESTUDIANTE, SEGUIR	N:M (Recursiva)	Creación de la tabla SEGUIR con id_seguidor y id_seguido (ambas referenciando a ESTUDIANTE).	Es la forma más limpia de implementar una red de seguidores sin crear campos redundantes, gestionando la conexión social.

5. Conclusiones del proceso

- **Dificultades y dudas:** Al inicio hubo confusión sobre cómo manejar la funcionalidad de "Seguir" a otro usuario, pero comprendimos que era una relación recursiva de muchos a muchos que requería su propia tabla para solucionarse lógicamente.
- **Aspectos relevantes:** Encontré gratificante en entender cómo funcionan las tablas de tipo para categorizar como catálogo los tipos extras dentro de cada identidad " (como ESTADO_ESTUDIANTE, TIPO_PUBLICACION, TIPO_MENTORIA), me ayudó a mantener la integridad de los datos, evitando que los usuarios escriban textos libres con errores ortográficos, o a la especulación.
- **Proceso de normalización:** Me asegure de que el modelo cumple hasta la Tercera Forma Normal (3FN), ya que todos los atributos dependen únicamente de su clave primaria y eliminamos cualquier dependencia extraña para evitar los datos repetitivos.